Super Top-Up Health Insurance — API Documentation

1. Get Super Top-Up Plans

Endpoint

GET https://policyplanner.com/health-insurance//companies/plans?policy=super_top_up

Method

`GET`

Query Parameter

Parameter	Type	Required	Value
`policy`	String	Yes	`super_top_up`

Example Request

GET https://policyplanner.com/health-insurance//companies/plans?policy=super_top_up

Success Response


```
{
  "success": true,
  "count": 7,
  "policy_filter": "super_top_up",
  "data": [
    {
      "api_type": "https://policyplanner.com/health-insurance/nic/1005/193/supertopup"
    },
    {
      "api_type": "https://policyplanner.com/health-insurance/hdfc/1008/194/supertopup"
    },
    {
      "api_type": "https://policyplanner.com/health-insurance/icici/1009/192/supertopup"
    },
    {
```

```
    "api_type": "https://policyplanner.com/health-insurance/tata/1011/195/supertopup"
   },
   {
    "api_type": "https://policyplanner.com/health-insurance/bajaj/1018/198/supertopup"
   },
   {
    "api_type": "https://policyplanner.com/health-insurance/united/1021/205/supertopup"
   },
   {
    "api_type": "https://policyplanner.com/health-insurance/iffco/1022/204/supertopup"
   }
  ]
}
```


---

2. Get Plans API Response

Field	Type	Description
`success`	Boolean	Indicates whether the request was successful
`count`	Integer	Total number of available Super Top-Up plans
`policy_filter`	String	Applied policy filter
`data`	Array	List of available plan APIs
`api_type`	String	Premium calculation API for the individual plan

Currently, the API returns **7 Super Top-Up Premium APIs**.

---

3. Super Top-Up Premium API

The `api_type` received from the Plans API must be called individually to calculate the premium for each insurance plan.

API Pattern

POST
https://policyplanner.com/health-insurance/{company}/{companyId}/{planId}/supertopup

Example

POST https://policyplanner.com/health-insurance/nic/1005/193/supertopup

Headers

Content-Type: application/json

---

4. Request Payload

```
{
  "coverAmount": 1000000,
  "age": 18,
  "sage": null,
  "c1age": null,
  "c2age": null,
  "c3age": null,
  "c4age": null
}
```

---

5. Request Parameters

Parameter	Type	Required	Description
`coverAmount`	Number	Yes	Selected Super Top-Up coverage amount
`age`	Number	Yes	Self / primary insured age
`sage`	Number / null	No	Spouse age
`c1age`	Number / null	No	Child 1 age
`c2age`	Number / null	No	Child 2 age
`c3age`	Number / null	No	Child 3 age
`c4age`	Number / null	No	Child 4 age

Unlike the standard Health Insurance Premium API, the Super Top-Up API **does not require `zone`** in the request payload.

---

6. Cover Amount

The `coverAmount` represents the Super Top-Up Sum Insured.

The selected value should be sent as a numeric value.

Example

```
{
  "coverAmount": 1000000
}
```

This represents:

10 Lakh

If the Super Top-Up module uses a predefined cover amount dropdown, the frontend should send the **numeric value**, not the display label.

---

7. Member Information

The API supports the following members:

age → Self / Primary Insured
sage → Spouse
c1age → Child 1
c2age → Child 2
c3age → Child 3
c4age → Child 4

Individual

```
{
  "coverAmount": 1000000,
  "age": 18,
  "sage": null,
  "c1age": null,
  "c2age": null,
  "c3age": null,
  "c4age": null
```

}

Self + Spouse

{
  "coverAmount": 1000000,
  "age": 35,
  "sage": 32,
  "c1age": null,
  "c2age": null,
  "c3age": null,
  "c4age": null
}

Family

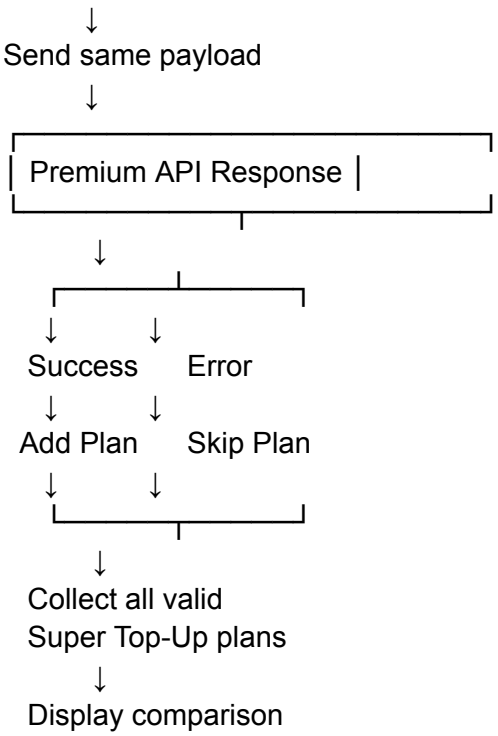
{
  "coverAmount": 1000000,
  "age": 35,
  "sage": 32,
  "c1age": 8,
  "c2age": 5,
  "c3age": null,
  "c4age": null
}

---

8. Premium Calculation Flow

The Super Top-Up API flow is:

User selects Super Top-Up
↓
Select Cover Amount
↓
Enter Member Ages
↓
GET Plans API
↓
policy=super_top_up
↓
Receive 7 api_type URLs
↓
Call every api_type API



---

9. Premium Success Response

When the premium is available, the API returns the complete Super Top-Up plan information.

Example


```
{
  "message": "National Insurance Super Top-Up Premium Calculated",
  "companyId": "1005",
  "planId": "193",
  "companyName": "National Insurance",
  "logoUrl": "national.png",
  "brochureUrl":
"https://policyplanner.com/super-top-up/assets/super-top-up-brochure/national-super-top-up.
pdf",
  "onePagerUrl":
"https://policyplanner.com/super-top-up/assets/st-onePager/national-mediclaim.jpeg",
  "planName": "Super Top Up Mediclaim policy",
  "coverAmount": 1000000,
  "adults": 1,
  "children": 0,
  "eldestActual": 18,
  "eldestLookup": 18,
  "premiums": [
    {
```

```
    "premium": 1838,
    "deductible": "500000"
   },
   {
    "premium": 1658,
    "deductible": "800000"
   }
  ],
  "members": [
   {
    "label": "self",
    "age": 18
   }
  ],
  "otherDetails": "...",
  "features": []
}
```


---

10. Premium Response Fields

Field	Type	Description
`message`	String	Premium calculation status/message
`companyId`	String	Insurance company ID
`planId`	String	Insurance plan ID
`companyName`	String	Insurance company name
`logoUrl`	String	Company logo
`brochureUrl`	String	Plan brochure URL
`onePagerUrl`	String	Plan one-pager URL
`planName`	String	Super Top-Up plan name
`coverAmount`	Number	Selected cover amount

`adults`	Number	Number of adult members
`children`	Number	Number of child members
`eldestActual`	Number	Actual age of eldest member
`eldestLookup`	Number	Age used for premium calculation
`premiums`	Array	Available premium/deductible combinations
`members`	Array	Member details
`otherDetails`	String/Object	Additional plan information
`features`	Array	Plan features

---

11. Premiums Object

The `premiums` array contains the premium for each available deductible.

Example


```
"premiums": [
 {
  "premium": 1838,
  "deductible": "500000"
 },
 {
  "premium": 1658,
  "deductible": "800000"
 }
]
```


Fields

Field	Type	Description
`premium`	Number	Premium payable for the deductible
`deductible`	String	Deductible amount applicable to the premium

Example Interpretation

Deductible: ₹5,00,000
Premium: ₹1,838

Deductible: ₹8,00,000
Premium: ₹1,658

Therefore, the UI should display each deductible/premium combination separately.

---

12. Members Object

The `members` array provides details of all insured members.

Example

```
"members": [
  {
    "label": "self",
    "age": 18
  }
]
```

Possible labels:

self
spouse
child1
child2
child3
child4

---

13. Other Details

The `otherDetails` field contains additional plan information such as:

```
{
  "Room Rent": "Actuals",
  "ICU Limit": "Actuals",
  "Pre Hospitalization": "60 days",
```

```
  "Post Hospitalization": "90 days",
  "Road Ambulance": "₹2,500 per event",
  "Air Ambulance": "Not covered",
  "Day Care": "Covered",
  "Domiciliary Treatment": "Not covered",
  "Health Checkup": "Not covered",
  "Organ Donor": "Covered",
  "Refill Restore Reload Recharge": "Not covered",
  "No Claim Bonus": "Not covered",
  "AYUSH Benefit": "Not covered",
  "AYUSH Pre & Post": "Not covered",
  "Initial Waiting Period": "30 days",
  "Specific Disease Waiting": "24 months",
  "Pre Existing Waiting": "48 months",
  "Multi Years Discount": "Not stated",
  "Hospital Cash": "Add-on",
  "Consumables Coverage": "Covered"
}
```


> Note: In the actual API response, this may be returned as a JSON-formatted **string**, so the frontend/backend may need to parse it before displaying individual details.

---

14. Features Response

The `features` array contains the plan's coverage, exclusions and add-ons.

Structure


```
{
  "id": 377,
  "company_id": "1005",
  "plan_id": "193",
  "includes": "In-patient Treatment",
  "excludes": "Pre-Existing Diseases",
  "addons": null,
  "created_at": "2025-11-25T08:04:23.000Z"
}
```


Feature Fields

Field	Type	Description

`id`	Number	Feature ID
`company_id`	String	Insurance company ID
`plan_id`	String	Insurance plan ID
`includes`	String	Coverage included
`excludes`	String	Exclusions
`addons`	String / null	Available add-on
`created_at`	DateTime	Feature creation date

---

15. Premium Not Available Response

If a particular company/plan does not have a premium for the selected member combination or cover amount, the API returns:

```
{
  "error": "No premiums found"
}
```

Handling

This should be treated as a **plan-level failure**.

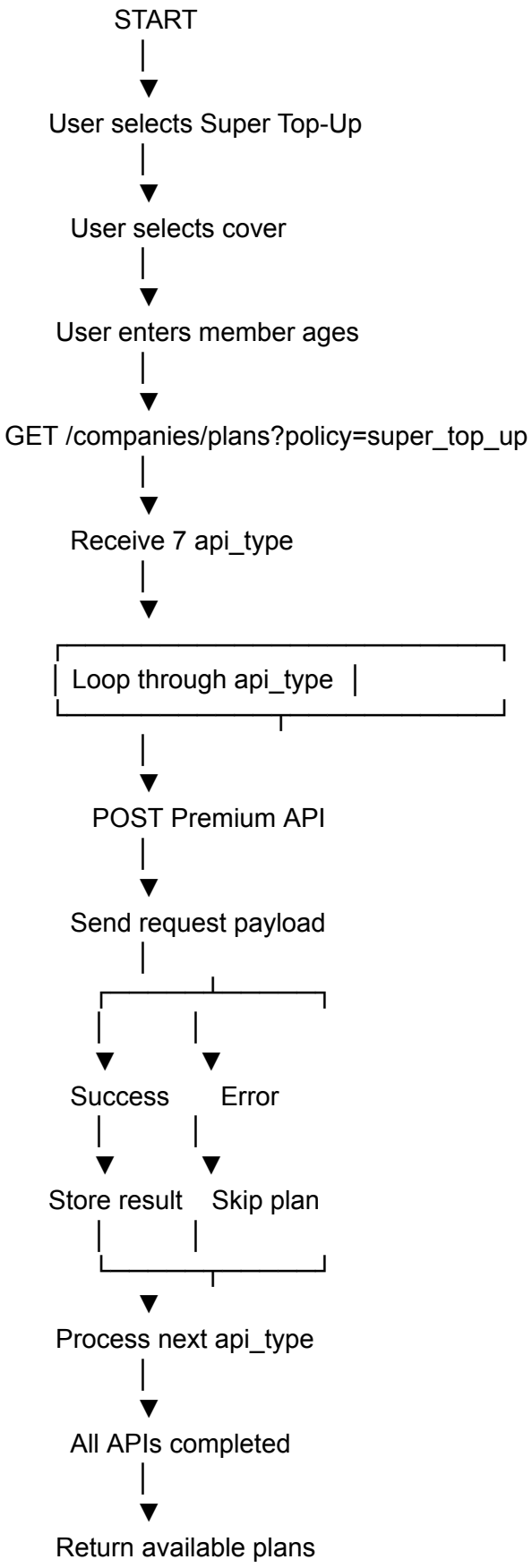
The application should:

Plan 1 → Premium available → Add result
Plan 2 → Premium available → Add result
Plan 3 → No premiums found → Skip
Plan 4 → Premium available → Add result
Plan 5 → No premiums found → Skip
Plan 6 → Premium available → Add result
Plan 7 → Premium available → Add result

The failure of one plan **must not stop the remaining Premium APIs from being called**.

---

16. Complete API Integration Flow





Display Premium Comparison

---

17. Example End-to-End Request

Step 1 — Get APIs

GET https://policyplanner.com/health-insurance//companies/plans?policy=super_top_up

Suppose it returns:


```
{
  "data": [
    {
      "api_type": "https://policyplanner.com/health-insurance/nic/1005/193/supertopup"
    },
    {
      "api_type": "https://policyplanner.com/health-insurance/hdfc/1008/194/supertopup"
    },
    {
      "api_type": "https://policyplanner.com/health-insurance/icici/1009/192/supertopup"
    }
  ]
}
```


Step 2 — Call First API

POST https://policyplanner.com/health-insurance/nic/1005/193/supertopup

Payload


```
{
  "coverAmount": 1000000,
  "age": 18,
  "sage": null,
  "c1age": null,
  "c2age": null,
  "c3age": null,
  "c4age": null
}
```

Step 3 — Call Remaining APIs

The **same payload** is sent to:

HDFC
ICICI
TATA
Bajaj
United
IFFCO

The only thing that changes is the `api_type` URL.

---

18. Final Result Structure

After calling all 7 APIs, the application should collect only successful responses.

Example:


```
[
 {
  "companyId": "1005",
  "planId": "193",
  "companyName": "National Insurance",
  "planName": "Super Top Up Mediclaim policy",
  "coverAmount": 1000000,
  "premiums": [
   {
    "premium": 1838,
    "deductible": "500000"
   },
   {
    "premium": 1658,
    "deductible": "800000"
   }
  ]
 },
 {
  "companyId": "1008",
  "planId": "194",
  "companyName": "HDFC",
  "planName": "HDFC Super Top Up",
  "coverAmount": 1000000,
  "premiums": []
```

```
  }
]
```

Plans returning:

```
{
  "error": "No premiums found"
}
```

should **not be included** in the final available-plan results.

---

19. API Summary

#	API	Method	Purpose
1	`/health-insurance//companies/plans?policy=super_top_up`	`GET`	Get all Super Top-Up plan Premium API URLs
2	`/health-insurance/{company}/{companyId}/{planId}/supertopup`	`POST`	Calculate Super Top-Up premium for each plan

Premium API Pattern

https://policyplanner.com/health-insurance/
{company}/
{companyId}/
{planId}/
supertopup

Current Available Plans

1. National Insurance
2. HDFC
3. ICICI
4. TATA
5. Bajaj
6. United
7. IFFCO

Key Integration Rules

1. **Do not hardcode the 7 Premium API URLs.** Fetch them dynamically from the Plans API.
2. Call **every** `api_type` returned by the Plans API.
3. Send the **same user-selected payload** to every Premium API.
4. `zone` is **not required** for Super Top-Up.
5. If a Premium API returns `"No premiums found"`, **skip that plan and continue**.
6. `premiums` can contain **multiple deductible/premium combinations**, so all combinations should be displayed/processed.
7. Use the numeric `coverAmount` value in the request.
8. Parse `otherDetails` if it is returned as a JSON string.